

Highlights

The Beggining of Control Revolution: Offset-free Koopman MPC

Patrik Valábek

- Research highlight 1
- Research highlight 2

The Beggining of Control Revolution: Offset-free Koopman MPC

Patrik Valábek

^a*STU, Redlinskeho 9, Bratislava, Slovakia*

Abstract

Keywords:

1. Introduction

2. Preliminaries and Notation

2.1. Offset-Free MPC

In this subsection, we will present a standard offset free model predictive control framework, introduced by Pannocchia and Rawlings (2003). This approach consist of state estimation - observer, target optimization and dynamic optimization. The whole approach is used to introduce integral action into model predictive control to help mitigate plant-model mismatch and ensure offset free control. If this compensation would not be used, the plant with this controller would not reach a desired reference (other than at origin), due to predictions using different model than actual plant.

The offset-free augmented system can be written in the following state-space form, including the disturbance input matrix B_d and disturbance output matrix C_d :

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k, \quad y_k = \begin{bmatrix} C & C_d \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} \quad (1)$$

where x_k is the original system state, d_k is the estimated constant disturbance, u_k is the control input, y_k is the output, A , B , and C are the original state-space matrices, B_d and C_d encode how the disturbance enters the state and output equations, and I is the identity matrix. Here, n_x , n_u , n_y , and n_d denote the state, input, output, and disturbance dimensions, respectively, so

that $x_k \in \mathbb{R}^{n_x}$, $u_k \in \mathbb{R}^{n_u}$, $y_k \in \mathbb{R}^{n_y}$, $d_k \in \mathbb{R}^{n_d}$, $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, $B_d \in \mathbb{R}^{n_x \times n_d}$, $C_d \in \mathbb{R}^{n_y \times n_d}$, and $I \in \mathbb{R}^{n_d \times n_d}$. For the offset free behavior, the number of the disturbances n_d must be lower or equal to the number of the measurements n_y . Also it is necessary to note that the number of offset free tracking is achievable only in the number of disturbances that are estimated. For the system where $n_d = n_y$ system and choice of the matrices B_d and C_d there is a necessary condition, that:

$$\text{rank} \begin{bmatrix} I - A & -B_d \\ C & C_d \end{bmatrix} = n_x + n_y. \quad (2)$$

When solving the MPC, both states and disturbance are approximated by observer.

2.2. Koopman Operator

The koopman operator theory was introduced by Koopman (1931) and further popularize by Mezić (2005). The theory enables to exactly model nonlinear systems linearly, by lifting its current states/measurements into infinite dimensional variables, where the governing operator for the evolving system is linear.

TODO: fancy smenši equation

Over the years many approximations of the koopman operator from the data were proposed, starting with connection of DMD to Koopman operator theory by Rowley et al. (2009), extended to the EDMD by Williams et al. (2015) and recently the neural networks were used to best approximate the lifting functions so the system behaves linearly. The algorithms were as well developed not only for autonomous systems but for the systems with control as well Kaiser et al. (2021) **TODO: elaborate half a sentence** . Koopman operator met the model predictive control in work of Korda and Mezić (2018), and in recent years almost every kind of MPC was transformed to accept some kind of Koopman Operator, mostly represented as an LTI system in the state space form. The offset free version is no exception, in the work of **TODO: Zotero, kukni papere, prosím ešte aj naskenuj ďalšie** .

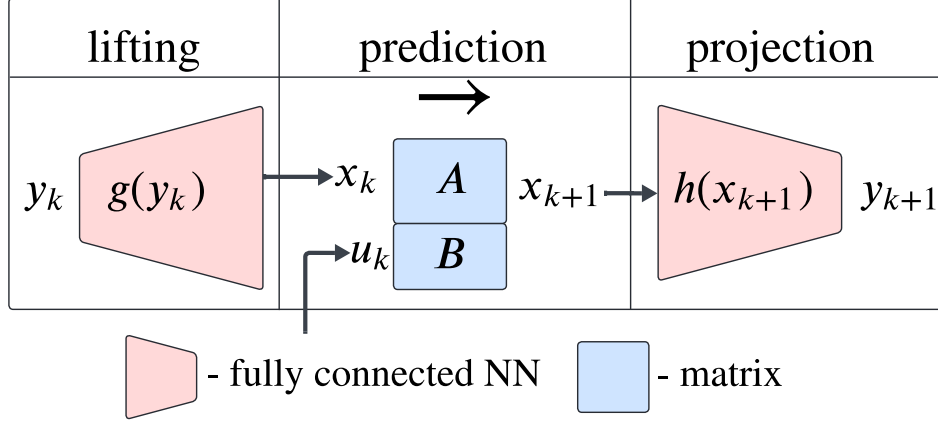


Figure 1: TODO:

For the purpose of this work we are using the Deep Koopman Approximation introduced by Lusch et al. (2018), transformed for the nonautonomous systems, as in Fig. 1. The control inputs u are used in the scheme directly, without any transformation to preserve the structure necessary for the use in MPC. We use two kinds of approximations of Koopman operator. Firstly the LTI approximation, where the projection function is linear (matrix C), as in Fig. 2 and second the approximation, where only the lifted system evolves linearly, as already shown in Fig. 1, but its projection is nonlinear (as it should be from the definition of Koopman operator).

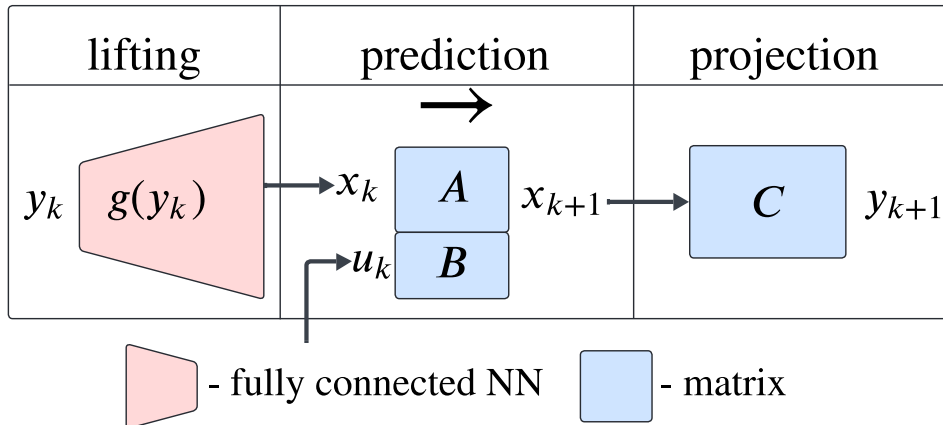


Figure 2: TODO:

3. Koopman MPC with Offset-Free

In this subsection, we will present a standard offset free model predictive control framework using approximation of Koopman operator as a linear time invariant (LTI) model. As observer is standard Kalman filter, we will discuss mainly remaining two optimization problems. As we use the LTI approximation of Koopman operator, we can use the standard MPC formulation with linear constraints and objective function. The control structure is shown in Figure 3.

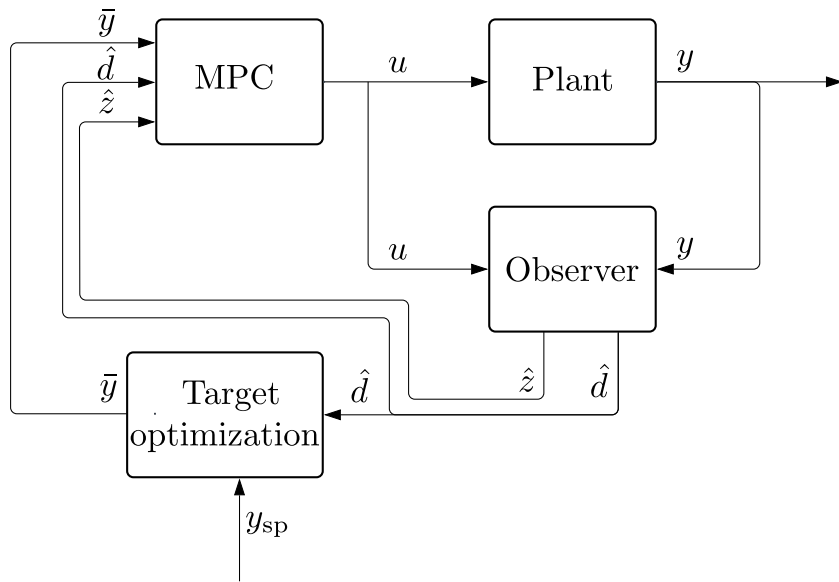


Figure 3: Basic closed-loop control structure.

The target optimization problem computes the steady-state targets $\bar{u}, \bar{z}, \bar{y}$ that minimizes the difference between reference y_{ref} and feasible output $\bar{y}$ in steady state. The problem is formulated as:

$$\min_{\bar{u}, \bar{z}, \bar{y}} (\bar{y} - y_{\text{ref}})^\top Q_y (\bar{y} - y_{\text{ref}}), \quad (3a)$$

$$\text{s.t.} \quad \bar{z} = A\bar{z} + B\bar{u}, \quad (3b)$$

$$\bar{y} = C\bar{z} + d, \quad (3c)$$

$$u_{\min} \leq \bar{u} \leq u_{\max}, \quad (3d)$$

$$y_{\min} \leq \bar{y} \leq y_{\max}, \quad (3e)$$

$$d = \hat{d}(t), \quad y_{\text{ref}} = y_{\text{sp}}(t), \quad (3f)$$

where $\bar{u} \in \mathbb{R}^{n_u}$, $\bar{z} \in \mathbb{R}^{n_z}$, and $\bar{y} \in \mathbb{R}^{n_y}$ are the steady-state input, lifted state, and output, respectively; $Q_y \succ 0$ is the output tracking weight matrix; $A \in \mathbb{R}^{n_z \times n_z}$, $B \in \mathbb{R}^{n_z \times n_u}$, $C \in \mathbb{R}^{n_y \times n_z}$ are the state-space matrices of the LTI Koopman model; $d = \hat{d}(t) \in \mathbb{R}^{n_y}$ is the additive output disturbance estimate provided by the Kalman filter; $y_{\text{ref}} = y_{\text{sp}}(t)$ is the output setpoint; and $u_{\min}, u_{\max}, y_{\min}, y_{\max}$ are the input and output constraint bounds.

Remark 1. For this

Given the steady-state target $(\bar{u}, \bar{z}, \bar{y})$, the MPC problem is solved at each sampling instant to compute the optimal input sequence over the horizon N :

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} (y_k - \bar{y})^\top Q_y (y_k - \bar{y}) + \sum_{k=0}^{N-1} \Delta u_k^\top Q_u \Delta u_k, \quad (4a)$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \quad (4b)$$

$$y_k = Cz_k + d, \quad k \in \mathbb{N}_0^{N-1}, \quad (4c)$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4d)$$

$$u_{\min} \leq u_k \leq u_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4e)$$

$$y_{\min} \leq y_k \leq y_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4f)$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t - T_s), \quad (4g)$$

where $\hat{z}(t) \in \mathbb{R}^{n_z}$ is the lifted state estimate provided by the Kalman filter; and $u_{-1} = u(t - T_s)$ is the input applied at the previous sampling instant, used to compute the first increment Δu_0 . Following the receding-horizon principle, only the first element u_0^* of the optimal sequence is applied to the plant.

4. Methodology

Main drawback that we observe in a current Koopman MPC (offset-free, robust, or any other), is that the optimization is done without fully leveraging knowledge of the lifted space. Of course we are predicting in the lifted space, but in objective function we are using original space, that is created derived as linear transformation of the lifted space $y_k = Cz_k$. This transformation is not linear by nature $y_k = h(z_k)$, but to use linear MPC, we think we had no other choice. Some papers are not using this transformation directly, but after closer look, they are usually using equivalent transformations, for example expanding lifted space with measurements, but dynamic in lifted space is linear, so transformations to achieve back this measurements is linear. We will show that we can use the lifted space directly in the objective function, showing how to handle the lifted space and transformation of an objective function. We can fully eliminate the necessity of the back transformation used in objective function in MPC, but we still need them to compute the target for the lifted space. In this case and case of constraints, we use a first order Taylor expansion, to better approximate the transformation.

4.1. Deep Koopman identification

Both prediction models used in this work are obtained with the deep Koopman architecture of Fig. 1, in which the encoder lifts the measured outputs, the lifted state evolves linearly, and the decoder projects the lifted state back to the outputs. Given an input-output trajectory of the plant, the model is unrolled over a window of N_r samples as

$$z_0 = g(y_0), \tag{5a}$$

$$z_{i+1} = Az_i + Bu_i, \quad i \in \mathbb{N}_0^{N_r-1}, \tag{5b}$$

$$\hat{y}_i = h(z_i), \quad i \in \mathbb{N}_0^{N_r}, \tag{5c}$$

where N_r is the length of the training window, i is the sample index within this window, y_i and u_i are the measured output and the applied input taken from the data, $z_i \in \mathbb{R}^{n_z}$ is the lifted state, and $\hat{y}_i \in \mathbb{R}^{n_y}$ is the predicted output. The encoder g and the decoder h are multilayer perceptrons, while the input map B is kept linear, so that the identified model is an LTI system in the lifted coordinates and can be used directly in the optimization problems (8) and (9).

The four terms of the training loss are the multi-step output prediction error over the whole window, the consistency between the unrolled lifted state and the lifted measurement, the one-step output prediction error, and the reconstruction error of the autoencoder, resulting in

$$\begin{aligned} \mathcal{L} = & \frac{w_y}{N_r - 1} \sum_{i=1}^{N_r-1} \|\hat{y}_i - y_i\|_2^2 + \frac{w_z}{N_r - 1} \sum_{i=1}^{N_r-1} \|z_i - g(y_i) - Bu_i\|_2^2 \\ & + w_1 \|\hat{y}_1 - y_1\|_2^2 + w_r \|\hat{y}_0 - y_0\|_2^2, \end{aligned} \quad (6)$$

where $\mathcal{L}$ is the loss minimized during training and w_y , w_z , w_1 , and w_r are the weights of the four terms, respectively. All terms are additionally averaged over the minibatch of training windows. The one-step term w_1 is fixed to one, so that only three weights have to be chosen, and the remaining weights are scaled relative to it.

When the decoder is restricted to a single matrix, $h(z) = Cz$, the identified model is the LTI model of the standard offset-free Koopman MPC of Sec. 3. When the decoder is a nonlinear map, the model is the one used by the proposed scheme, and a constant output matrix is obtained afterwards by linear least squares from the lifted trajectory of the training data. This matrix is not used in the objective function or in the constraints of (9); it only provides the initial output matrix of the observer of Sec. 4.6, which is replaced by the Jacobian $H(z_{LP})$ at every subsequent sampling instant.

The lifted dimension n_z , the depth and the width of the encoder and the decoder, the activation function, the weights in (6), the window length N_r , the minibatch size, and the learning rate are not tuned by hand. Each candidate configuration is trained, the resulting model is inserted into the closed loop, and the achieved value of the closed-loop objective function is used as the score of the configuration; the search itself is carried out with the tree-structured Parzen estimator of Optuna Akiba et al. (2019). Scoring a model by the closed-loop performance rather than by the open-loop prediction error is a deliberate choice, because the two are only loosely related, as the results of Sec. 5 and Sec. 6 show. The same criterion is used to select the order and the horizons of the subspace model that serves as the baseline, so that no model is favored by the way its parameters were chosen. Note that this procedure is not a contribution of this work and any comparable model selection scheme can replace it.

4.2. Linearization of Output Mapping

For the control problem, we are not able to completely eliminate the necessity of the transformation from the lifted space back to the original space. For this, we linearize the output mapping with first order Taylor expansion. For the purpose of this work we experimented with linearization at two different points: linearization at the previous target and linearization at the current state estimate.

$$y_k = h(z_k) \approx h(\bar{z}(t - T_s)) + J_{\bar{z}(t-T_s)}(z_k - \bar{z}(t - T_s)), \quad (7a)$$

$$y_k = h(z_k) \approx h(\hat{z}(t)) + J_{\hat{z}(t)}(z_k - \hat{z}(t)), \quad (7b)$$

where $h : \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{n_y}$ is the nonlinear output mapping; $\bar{z}(t - T_s)$ is the steady-state lifted state computed by the target optimizer at the previous sampling instant; $J_{\bar{z}(t-T_s)}$ and $J_{\hat{z}(t)}$ are the Jacobians of h evaluated at $\bar{z}(t - T_s)$ and $\hat{z}(t)$, respectively.

4.3. Target Optimization

The objective function of target estimation is the same as in the standard offset-free MPC. The main difference is in

The proposed target optimization problem is formulated as:

$$\min_{\bar{u}, \bar{z}, \bar{y}} (\bar{y} - y_{\text{ref}})^\top Q_y (\bar{y} - y_{\text{ref}}), \quad (8a)$$

$$\text{s.t.} \quad \bar{z} = A\bar{z} + B\bar{u}, \quad (8b)$$

$$\bar{y} = H(z_{\text{LP}})\bar{z} + h(z_{\text{LP}}) - H(z_{\text{LP}})z_{\text{LP}} + d, \quad (8c)$$

$$u_{\min} \leq \bar{u} \leq u_{\max}, \quad (8d)$$

$$y_{\min} \leq \bar{y} \leq y_{\max}, \quad (8e)$$

$$d = \hat{d}(t), \quad y_{\text{ref}} = y_{\text{sp}}, \quad z_{\text{LP}} = \bar{z}_{-1}, \quad (8f)$$

where $H(z_{\text{LP}})$ is the Jacobian of the transformation $h(z_{\text{LP}})$. z_{LP} is a linearization point, the point where we approximate our function with first order Taylor expansion. We propose, that this point should be as close as possible to the $\bar{z}$. Therefore we are using the previously calculated target $\bar{z}_{-1}$. This allows us to better approximate the transformation of the lifted space to the original space and more precise target optimization.

4.4. MPC

The proposed MPC problem is formulated as:

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} (z_k - \bar{z})^\top Q_z (z_k - \bar{z}) + \sum_{k=0}^{N-1} \Delta u_k^\top Q_u \Delta u_k, \quad (9a)$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \quad (9b)$$

$$y_k = H(z_{\text{LP}})z_k + h(z_{\text{LP}}) - H(z_{\text{LP}})z_{\text{LP}} + d, \quad k \in \mathbb{N}_0^{N-1}, \quad (9c)$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9d)$$

$$u_{\min} \leq u_k \leq u_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9e)$$

$$y_{\min} \leq y_k \leq y_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9f)$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t - T_s) \quad (9g)$$

where $H(z_0)$ is the Jacobian of the transformation $h(z_0)$ at the point z_0 . This linearization point is chosen to best approximate the surroundings of current operational point. The first order Taylor expansion is done to receive y_k , which is used only in the constraints of the system. In objective function, we use a tuning matrix Q_z to penalize the difference from lifted space target. We propose to derive this tuning matrix as follows:

$$Q_z = H(\bar{z})^\top Q_y H(\bar{z}). \quad (10)$$

This allows us to use and tune our problem using matrix Q_y , which is convenient and easily tuned. This is a significant change from the standard Koopman MPC, where the objective function is defined in the original space.

4.5. Closed-Loop Control Structure

The close loop structure is similar to the previous with the few tweaks. There is an addition of the linearization block. Also there is a time varying Kalman filter as an observer and the optimization problems have a different formulation.

For the linearization at previous target the algorithm is:

Algorithm 1 Closed-loop control with nonlinear output mapping with linearization at previous target

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

- 1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$; solve (8) with $z_{\text{LP}} = \hat{z}(0)$ and $\hat{d}(0)$ to obtain $\bar{z}_{-1}$
 - 2: **loop**
 - 3: **Measurement:** Obtain $y(t)$ from the plant
 - 4: **Linearization:** Compute $H(\bar{z}_{-1})$ and $h(\bar{z}_{-1})$ — output mapping linearized at $z_{\text{LP}} = \bar{z}_{-1}$
 - 5: **Observer:** Run the time-varying Kalman filter with $y(t)$ and $u(t - T_s)$ to obtain updated estimates $\hat{z}(t)$ and $\hat{d}(t)$
 - 6: **Target estimation:** Solve (8) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$; set $\bar{z}_{-1} \leftarrow \bar{z}(t)$
 - 7: **MPC:** Solve (9) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
 - 8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
 - 9: **end loop**
-

where $g : \mathbb{R}^{n_y} \rightarrow \mathbb{R}^{n_z}$ is the Koopman lifting function that provides the initial lifted-state estimate, and $\hat{d}(0) = 0$ initialises the disturbance estimate at zero.

For the linearization at current estimate, it is:

Algorithm 2 Closed-loop control with nonlinear output mapping (linearization at current state estimate)

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

- 1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$
 - 2: **loop**
 - 3: **Measurement:** Obtain $y(t)$ from the plant
 - 4: **Observer:** Run the time-varying Kalman filter with $y(t)$ and $u(t - T_s)$ to obtain updated estimates $\hat{z}(t)$ and $\hat{d}(t)$
 - 5: **Linearization:** Compute $H(\hat{z}(t))$ and $h(\hat{z}(t))$ — output mapping linearized at $z_{LP} = \hat{z}(t)$ **TODO: when this is happening?**
 - 6: **Target estimation:** Solve (8) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$; set $\bar{z}_{-1} \leftarrow \bar{z}(t)$
 - 7: **MPC:** Solve (9) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
 - 8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
 - 9: **end loop**
-

4.6. State Estimation - Time Varying Kalman Filter

The observer estimates the augmented state $\tilde{z}_k = [z_k^\top, d_k^\top]^\top$ from the noisy output y_k and the applied input u_k . The augmented linear dynamics are identical to (1) with x_k replaced by the lifted Koopman state z_k :

$$\tilde{z}_{k+1} = \tilde{A} \tilde{z}_k + \tilde{B} u_k, \quad \tilde{A} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \quad (11)$$

where $\tilde{z}_k \in \mathbb{R}^{n_z+n_d}$ is the augmented state stacking the lifted Koopman state $z_k \in \mathbb{R}^{n_z}$ and the output disturbance estimate $d_k \in \mathbb{R}^{n_d}$, $\tilde{A} \in \mathbb{R}^{(n_z+n_d) \times (n_z+n_d)}$ and $\tilde{B} \in \mathbb{R}^{(n_z+n_d) \times n_u}$ are the augmented system matrices.

Because the output map is nonlinear, the measurement equation is re-linearised at each sampling instant around a chosen linearisation point z_{LP} :

$$y_k \approx \tilde{C}_k \tilde{z}_k + c_k, \quad \tilde{C}_k = [H(z_{LP}), C_d], \quad c_k = h(z_{LP}) - H(z_{LP}) z_{LP}, \quad (12)$$

where $\tilde{C}_k \in \mathbb{R}^{n_y \times (n_z+n_d)}$ is the time-varying output matrix formed by stacking the Jacobian $H(z_{LP}) \in \mathbb{R}^{n_y \times n_z}$ of h at the linearisation point and the disturbance output matrix C_d , and $c_k \in \mathbb{R}^{n_y}$ is the linearisation offset.

With the linear prediction model (11) and the time-varying output matrix (12), the filter runs the standard predict–update recursion at each step k :

Predict:

$$\hat{\tilde{z}}_{k|k-1} = \tilde{A} \hat{\tilde{z}}_{k-1|k-1} + \tilde{B} u_{k-1}, \quad (13a)$$

$$P_{k|k-1} = \tilde{A} P_{k-1|k-1} \tilde{A}^\top + Q, \quad (13b)$$

where $\hat{\tilde{z}}_{k|k-1} \in \mathbb{R}^{n_z+n_d}$ is the prior augmented-state estimate, $P_{k|k-1} \in \mathbb{R}^{(n_z+n_d) \times (n_z+n_d)}$ is the prior error covariance, and $Q \succeq 0$ is the process noise covariance matrix.

Update:

$$S_k = \tilde{C}_k P_{k|k-1} \tilde{C}_k^\top + R, \quad (14a)$$

$$K_k = P_{k|k-1} \tilde{C}_k^\top S_k^{-1}, \quad (14b)$$

$$\hat{\tilde{z}}_{k|k} = \hat{\tilde{z}}_{k|k-1} + K_k \left(y_k - \tilde{C}_k \hat{\tilde{z}}_{k|k-1} - c_k \right), \quad (14c)$$

$$P_{k|k} = \left(I - K_k \tilde{C}_k \right) P_{k|k-1}, \quad (14d)$$

where $S_k \in \mathbb{R}^{n_y \times n_y}$ is the innovation covariance, $K_k \in \mathbb{R}^{(n_z+n_d) \times n_y}$ is the Kalman gain, $\hat{\tilde{z}}_{k|k}$ is the posterior augmented-state estimate whose upper n_z entries give $\hat{z}(t)$ and lower n_d entries give $\hat{d}(t)$, $P_{k|k}$ is the posterior error covariance, and $R \succ 0$ is the measurement noise covariance matrix.

4.7. Block Diagonal Deep Koopman Transformation

TODO: becare notation conflict zbar During our work, we encounter a several problems with the suboptimal or infeasible solutions using Koopman MPC using Gurobi solver. For the enhancement of the performance, we are proposing a block diagonal structure of the matrix A . This transformation can be done after obtaining the Koopman operator, with finding the transformation matrix T that transforms the Koopman operator to the block diagonal form:

$$\bar{A} = T^{-1} A T. \quad (15)$$

This transformation is done by finding the eigenvalues and eigenvectors of the Koopman operator, and then grouping them into blocks. The transformation matrix T is constructed from the eigenvectors, and the block diagonal form $\bar{A}$ is obtained by applying the transformation to the original Koopman operator A . Using this approach, we had to modify the rest of the model accordingly, using the transformation matrix T to transform the input and output matrices B and C as well as the lifted states z :

$$\bar{B} = T^{-1} B, \quad \bar{C} = C T, \quad \bar{z}_k = T^{-1} z_k. \quad (16)$$

When computing the Jacobian, we are transforming the original Jacobian $H(z_k)$ to the block diagonal form $\bar{H}(z_k)$ using the transformation matrix T :

$$\bar{H}(z_k) = H(z_k)T = H(T\bar{z}_k)T. \quad (17)$$

For fair comparison, this transformation is applied to every identified model through this work, not only for Koopman models. The effects of this transformation are discussed in the result section of this work.

5. Identification

In this section, we introduce the benchmark process, the data collected from it, and the three prediction models whose closed-loop performance is compared in Sec. 6. All models are identified from the same data set with the same scaling, and the parameters reported here are the final ones used in every closed-loop experiment of this work.

5.1. Reactor–separator process

The considered plant consists of two continuous stirred tank reactors in series followed by a flash separator, with a recycle stream from the separator back to the first reactor. The plant structure is the one of Liu et al. (2009) and the kinetic, thermodynamic, and geometric parameters are those reported by Li and Swartz (2019). Two consecutive first-order reactions $A \rightarrow B \rightarrow C$ take place in both reactors, where B is the desired product and C an undesired by-product. The plant is therefore a selectivity problem, since additional heat increases the conversion of the reactant A but also the loss of the product B into C.

The plant has nine states, namely the mass fractions of A and B and the temperature in each of the three vessels, and the third vessel is the separator, in which no reaction occurs. The manipulated variables are the three heat duties Q_1 , Q_2 , Q_3 , the fresh feed flow rates F_{10} and F_{20} to the first and the second reactor, and the recycle flow rate F_r . The measured and controlled outputs are the three vessel temperatures T_1 , T_2 , T_3 and the bottoms quality x_{B3} , which is the mass fraction of B leaving the separator. The vessel inventories are constant, so the internal flow rates follow from the manipulated ones. The nominal operating point and the box within which the inputs are moved during identification are listed in Table 1.

Two mechanisms make the plant nonlinear. The first one is the Arrhenius dependence of both reaction rates on the vessel temperatures, and the

Table 1: Manipulated variables of the reactor–separator plant, with the nominal operating point of Li and Swartz (2019) and the box used for the identification experiment.

Variable	Description	Nominal	Range
Q_1, Q_2, Q_3 [kJ/s]	Heat duties of the three vessels	10.0	[0.0, 25.0]
F_{10} [m ³ /s]	Fresh feed to the first reactor	8.3	[3.0, 16.0]
F_{20} [m ³ /s]	Fresh feed to the second reactor	0.5	[0.1, 3.0]
F_r [m ³ /s]	Recycle from the separator	4.0	[0.5, 10.0]

second one is the relative-volatility flash of the separator, which makes the composition of the recycle stream a nonlinear function of the composition in the separator and closes a nonlinear loop back to the first reactor. As a consequence, the bottoms quality is not monotone in the supplied heat: the yield of B reaches a peak and then decreases again as more of the product is converted into C. A model identified around a single operating point can therefore reproduce only one branch of this behavior, which is the reason for choosing this plant as the benchmark. The plant equations are integrated with the explicit Runge–Kutta solver of SciPy, and the sampling period used for identification and for control is $T_s = 1$ s.

5.2. Identification data

The excitation is a sequence of 100 piecewise-constant input steps, each held for 180 s, which gives 18 000 samples of input–output data. All six inputs are stepped within the box of Table 1, and the levels of the heat duties are paired with the fresh feed flow rate so that the excitation crosses the yield peak of the bottoms quality instead of staying on one of its branches. Over the whole data set the temperatures cover the range from 325 K to 393 K and the bottoms quality the range from 0.42 to 0.77.

Zero-mean Gaussian measurement noise is added to all four outputs, with a standard deviation equal to 1% of the range of the respective output, that is 0.494 K, 0.508 K, and 0.556 K for the three temperatures and 0.0033 for the bottoms quality. The data set is split into the first 12 960 samples used for training, the following 2880 samples used for validation and early stopping, and the remaining 2160 samples, which are never seen during identification and serve as the test data. Inputs and outputs are standardized to zero mean and unit variance using the statistics of the training split only, and all three models are identified in these coordinates, so that the four outputs contribute to the identification criteria on a comparable scale.

5.3. Subspace model

The baseline model is identified with the N4SID algorithm Van Overschee and De Moor (2012) as implemented in the SIPPY package Armenise et al. (2018), with the direct input–output term fixed to zero. The final model has the order 17 and the past and future horizons equal to 57, no centering is applied to the data, and no stability constraint is imposed during identification, which results in the spectral radius of 0.9897. Identifying this model takes 2.6 s on a Mac mini with an Apple M4 Pro processor.

5.4. Koopman models

Two Koopman models are trained with the loss (6) on the same data. The first one restricts the decoder to a single matrix and thus provides the LTI model required by the standard offset-free Koopman MPC of Sec. 3. Its lifted dimension is $n_z = 19$ and its encoder is a network with one hidden layer of four GELU units, which gives 666 trainable parameters in total and the spectral radius of 0.9337. The second one keeps the nonlinear decoder and is the model used by the proposed scheme. Its lifted dimension is $n_z = 26$, the encoder has the widths 4–7–14–26 and the decoder the mirrored widths 26–14–7–4, both with ELU activations, which gives 1884 trainable parameters and the spectral radius of 0.9622.

Both models are trained with the Adam optimizer in PyTorch Paszke et al. (2019) through the Neuromancer library Drgoña et al. (2023), on mini-batches of 40 windows of $N_r = 120$ samples, for at most 4000 epochs, with early stopping after 300 epochs without an improvement of the validation loss and a warm-up of 100 epochs. The learning rate is $9.9 \cdot 10^{-4}$ for the model with the linear decoder and $8.0 \cdot 10^{-4}$ for the model with the nonlinear decoder, and the loss weights (w_y, w_z, w_r) of (6) are (1.14, 2.15, 4.84) and (17.38, 1.59, 2.28), respectively. Training took 131 s, terminated by early stopping after 2937 epochs, for the model with the linear decoder, and 239 s for the model with the nonlinear decoder, on the same Mac mini with an Apple M4 Pro processor. As stated in Sec. 4.7, all three models are transformed into the real block-diagonal form before they are used in any control problem.

5.5. Open-loop accuracy on the test data

To compare the models on the data that none of them has seen, each model is initialized from the first sample of the test split and then run in open loop over the whole test window, driven only by the recorded inputs

and without any state correction. The prediction errors are evaluated against the noise-free outputs of the plant, which are available because the plant is simulated, so that the reported numbers reflect the model error alone and are not inflated by the measurement noise. Fig. 4 shows the first four input steps of this window and Table 2 summarizes the accuracy over the whole window.

Table 2: Open-loop prediction accuracy on the test data, measured by the mean absolute error (MAE) against the noise-free plant outputs. The last column is the sum of the mean absolute errors of the four standardized outputs, which is the criterion on which the models are directly comparable.

Model	T_1 [K]	T_2 [K]	T_3 [K]	x_{B3} [-]	Σ MAE [-]
N4SID, order 17	3.79	3.32	3.42	0.0193	1.21
Koopman, linear h , $n_z = 19$	4.41	3.93	4.10	0.0224	1.43
Koopman, nonlinear h , $n_z = 26$	0.47	0.50	0.61	0.0034	0.19

The Koopman model with the nonlinear decoder is significantly more accurate than both linear alternatives, and it reduces the sum of the mean absolute errors by approximately 85% with respect to the N4SID model. Its prediction error is of the same magnitude as the standard deviation of the measurement noise on all four outputs, which means that the model captures the plant dynamics down to the resolution of the data. As can be seen in Fig. 4, the two linear models follow the direction of every response but consistently miss the steady-state values reached after a step, and the deviation is largest for the bottoms quality, which is exactly the output whose dependence on the inputs is not monotone. The nonlinear decoder, in contrast, tracks both the transients and the steady states.

The Koopman model with the linear decoder is not better than the subspace model in open loop; its error is approximately 18% higher than that of the N4SID model, even though its lifted dimension exceeds the order of the subspace model. Restricting the decoder to a matrix removes most of the benefit of the lifting, since the lifted state then has to be a linear image of the outputs and the whole model remains an input–output map of an LTI system, for which subspace identification is a mature and well-tuned tool. This model is nevertheless retained in the comparison, because it is the model that the standard offset-free Koopman MPC formulation of Sec. 3 requires, and it therefore isolates the effect of the nonlinear output mapping

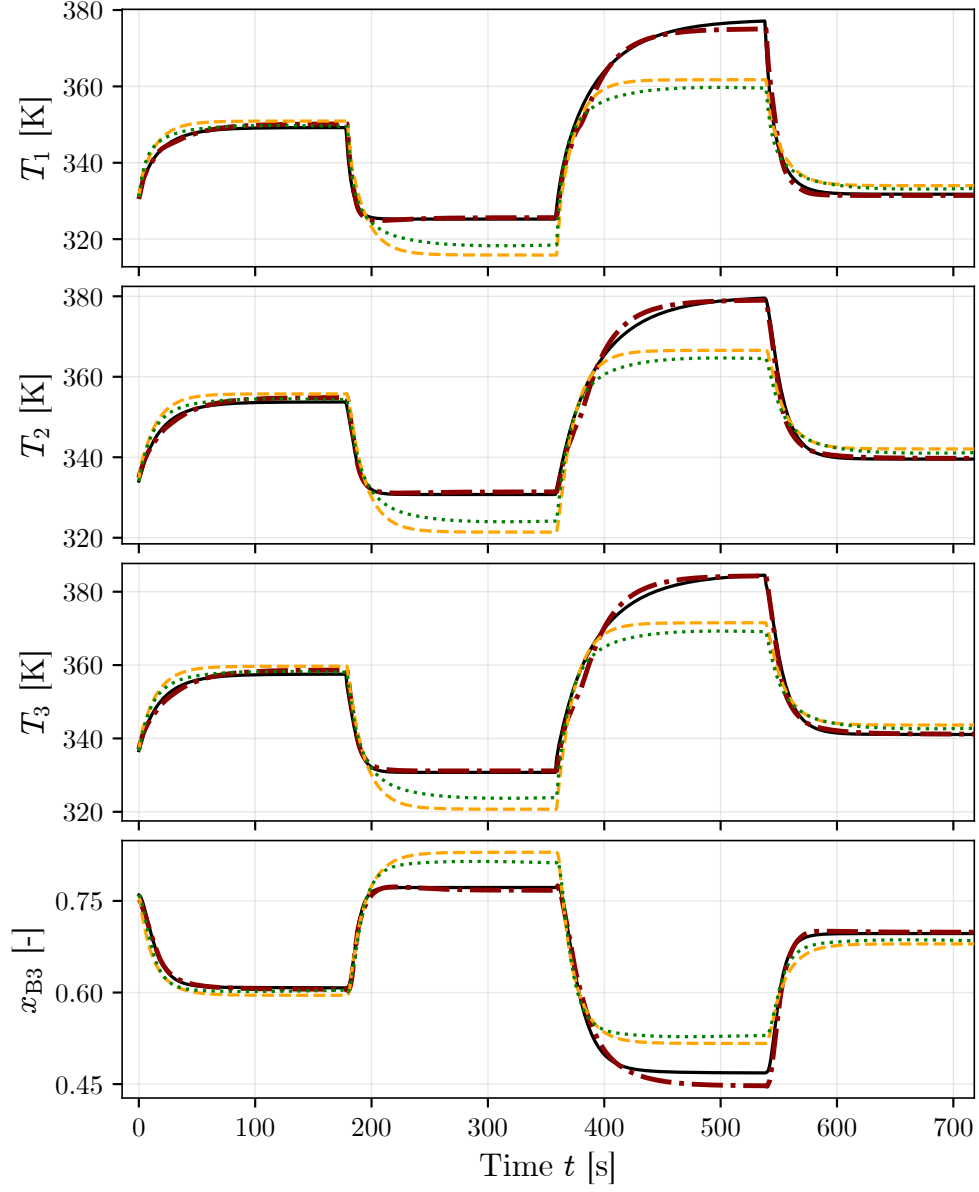


Figure 4: Open-loop predictions of the identified models on the first four input steps of the test data. The black solid line is the noise-free plant, the dash-dotted red line is the Koopman model with the nonlinear decoder, the dashed orange line is the Koopman model with the linear decoder, and the dotted green line is the N4SID model.

proposed in Sec. 4.1 from the effect of the lifting itself.

6. Control Results

In the following, we make precise the output mappings sketched in the notes and list the combinations (setups) evaluated in the simulations.

6.1. The effect of block-diagonal transformation

After this transformation we are able to enhance the performance in optimization as can be seen in Table 3. This table is done using the formulation of Koopman MPC which can be seen in Sec. 3. The table shows the decrease in a computational times. Also with this formulation we are able to obtain optimal solution 100% time during simulation. Contrary, the full Koopman MPC formulation is able to obtain optimal solution only 26.7% of the time with the rest of the time giving warning, that the solution is suboptimal. We can see also increase in a control performance, which is measured by the objective function value, which is in optimal case around 6% lower. Also the settling time for both levels is lower.

Table 3: Comparison of structure of problem to solver time and control performance.

Structure	Time TE/MPC	Optimal? [%]	OBJ	ST h_1	ST h_2
Full	0.2392 / 2.3884	26.7	100.0	43	48
Block Diagonal	0.2191 / 1.6204	100.0	94.1	25	31

The second option how to obtain the block diagonal structure is to learn the Koopman operator directly in the block diagonal form. This can be done by limiting the number of learnable parameters in a Koopman matrix during training to those that are in the block diagonal form. This also speeds the training as less parameters are learned.

To obtain an optimal performance, which is necessary for the real time control, we are proposing to use the block diagonal structure of the Koopman operator. There is no necessity to use the full Koopman operator, as the block diagonal structure is able to capture the same dynamics of the system and favors the optimization performance. The way this structure is obtained, during or after the training, is not important, as long as the Koopman operator is in the block diagonal form.

6.2. Simulation setups

6.3. Simulation Results for Two Reactor System

We index target mappings by T1–T3 and dynamic mappings by D1–D4 as in (??)–(??). Table 4 compares previous diagonal entries (CT, T2D2, T3D3) with reruns from the current scripts.

Table 4: Two-reactor CSTR: normalized closed-loop objective with NMPC benchmark 100. Diagonal combinations CT ($T1 \times D1$), T2D2 ($T2 \times D2$), and T3D3 ($T3 \times D3$). *Previous*: entries from the earlier full 3×4 TM/DM matrices. *Updated*: $100 J/J_{\text{NMPC}}$ from `control_cstr` (CT.py, T2D2.py, T3D3.py) with the same weighting as tuning I; em dashes mark tunings II–III not yet rerun.

Tuning	Previous			Updated		
	CT	T2D2	T3D3	CT	T2D2	T3D3
I: $Q_y = \text{diag}(5)$, $Q_u = \text{diag}(0.1)$	365.0	251.5	265.1	314.9	260.5	285.7
II: $Q_y = Q_u = I$	184.5	146.2	153.9	—	—	—
III: $Q_y = \text{diag}(0.1)$, $Q_u = \text{diag}(5)$	130.4	112.1	116.1	—	—	—

7. Discussion

8. Conclusion

Appendix A. System Models

In this appendix, we provide detailed mathematical descriptions of the two benchmark nonlinear systems used to evaluate the proposed offset-free Koopman MPC schemes: a two-tank system with hydraulic flows and a two-reactor CSTR series with recycle and parallel reactions.

Appendix A.1. Two Reactor System

The two-reactor system consists of two continuous stirred tank reactors (CSTRs) in series with a recycle loop from the second reactor back to the first. Each reactor is equipped with jacket cooling. The system models a classic selectivity problem with parallel competing reactions on reactant A.

Appendix A.1.1. Chemical Reactions

Three reaction pathways are considered:



where the temperature-dependent rate constants follow Arrhenius kinetics:

$$k_i(T) = k_{i,0} \exp\left(-\frac{E_i}{RT}\right), \quad i = 1, 2, 3. \quad (\text{A.4})$$

The autocatalytic shunt reaction (R3) occurs only in the first reactor.

Appendix A.1.2. Dynamic Model

The system has eight states:

$$x = [C_{A,1} \quad T_1 \quad C_{A,2} \quad T_2 \quad C_{B,1} \quad C_{B,2} \quad C_{U,1} \quad C_{U,2}]^\top, \quad (\text{A.5})$$

where $C_{A,i}$, $C_{B,i}$, and $C_{U,i}$ denote concentrations of species A, B, and U in reactor i , and T_i is the temperature. The four control inputs are:

$$u = [F \quad L \quad T_{c,1} \quad T_{c,2}]^\top, \quad (\text{A.6})$$

with F the fresh feed flow rate, L the recycle flow rate, and $T_{c,i}$ the jacket coolant temperatures.

The continuous-time dynamics are:

$$\frac{dC_{A,1}}{dt} = \frac{1}{V_1} [C_{A,0} F + L C_{A,2} - (F + L) C_{A,1}] - 2 r_{1,1} - r_{2,1} - r_{3,1}, \quad (\text{A.7a})$$

$$\frac{dC_{B,1}}{dt} = \frac{1}{V_1} [C_{B,0} F + L C_{B,2} - (F + L) C_{B,1}] + r_{1,1} + r_{3,1}, \quad (\text{A.7b})$$

$$\frac{dC_{U,1}}{dt} = \frac{1}{V_1} [C_{U,0} F + L C_{U,2} - (F + L) C_{U,1}] + r_{2,1}, \quad (\text{A.7c})$$

$$\begin{aligned} \frac{dT_1}{dt} = & \frac{1}{V_1} [T_0 F + L T_2 - (F + L) T_1] - \frac{U_1 A_1}{V_1 \rho c_p} (T_1 - T_{c,1}) \\ & + \frac{1}{\rho c_p} [(-\Delta H_1) r_{1,1} + (-\Delta H_2) r_{2,1} + (-\Delta H_3) r_{3,1}], \end{aligned} \quad (\text{A.7d})$$

$$\frac{dC_{A,2}}{dt} = \frac{F + L}{V_2} [C_{A,1} - C_{A,2}] - 2 r_{1,2} - r_{2,2}, \quad (\text{A.7e})$$

$$\frac{dC_{B,2}}{dt} = \frac{F + L}{V_2} [C_{B,1} - C_{B,2}] + r_{1,2}, \quad (\text{A.7f})$$

$$\frac{dC_{U,2}}{dt} = \frac{F + L}{V_2} [C_{U,1} - C_{U,2}] + r_{2,2}, \quad (\text{A.7g})$$

$$\begin{aligned} \frac{dT_2}{dt} = & \frac{F + L}{V_2} [T_1 - T_2] - \frac{U_2 A_2}{V_2 \rho c_p} (T_2 - T_{c,2}) \\ & + \frac{1}{\rho c_p} [(-\Delta H_1) r_{1,2} + (-\Delta H_2) r_{2,2}], \end{aligned} \quad (\text{A.7h})$$

where $r_{j,i}$ denotes the rate of reaction j in reactor i . The outputs are all eight states:

$$y = x. \quad (\text{A.8})$$

The system is discretized with sampling time $T_s = 1$ s using a Runge–Kutta integration scheme. Physical parameters, kinetic constants, and operational constraints are listed in Tables A.5–A.7.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631.

Table A.5: CSTR series physical and thermodynamic parameters.

Parameter	Description	Value
V_1	Volume of reactor 1	$1.0 \times 10^{-3} \text{ m}^3$
V_2	Volume of reactor 2	$2.0 \times 10^{-3} \text{ m}^3$
$U_1 A_1$	Heat transfer coefficient $\times$ area (R1)	$0.461 \text{ kJ}/(\text{s}\cdot\text{K})$
$U_2 A_2$	Heat transfer coefficient $\times$ area (R2)	$0.732 \text{ kJ}/(\text{s}\cdot\text{K})$
ρ	Liquid density	$1050 \text{ kg}/\text{m}^3$
c_p	Specific heat capacity	$3.766 \text{ kJ}/(\text{kg}\cdot\text{K})$
R	Universal gas constant	$8.3145 \times 10^{-3} \text{ kJ}/(\text{mol}\cdot\text{K})$
$C_{A,0}$	Feed concentration of A	$97.35 \text{ mol}/\text{m}^3$
$C_{B,0}, C_{U,0}$	Feed concentrations of B, U	$0.0 \text{ mol}/\text{m}^3$
T_0	Feed temperature	298.0 K

Table A.6: Arrhenius kinetic parameters for reactions R1–R3.

Reaction	Parameter	Pre-exponential	Activation Energy	Heat of Reaction
R1: $2 \text{ A} \rightarrow \text{B}$	$k_{1,0}, E_1, \Delta H_1$	$1.0 \times 10^5 \text{ m}^3/(\text{mol}\cdot\text{s})$	$45.0 \text{ kJ}/\text{mol}$	$60.0 \text{ kJ}/\text{mol}$
R2: $\text{A} \rightarrow \text{U}$	$k_{2,0}, E_2, \Delta H_2$	$9.8 \times 10^9 \text{ s}^{-1}$	$70.0 \text{ kJ}/\text{mol}$	$40.0 \text{ kJ}/\text{mol}$
R3: $\text{A} + \text{B} \rightarrow 2 \text{ B}$	$k_{3,0}, E_3, \Delta H_3$	$5.0 \times 10^4 \text{ m}^3/(\text{mol}\cdot\text{s})$	$55.0 \text{ kJ}/\text{mol}$	$60.0 \text{ kJ}/\text{mol}$

Armenise, G., Bacci di Capaci, R., Vaccari, M., Pannocchia, G., 2018. An open-source system identification package for multivariable processes, in: 2018 UKACC 12th International Conference on Control (CONTROL). doi:10.1109/CONTROL.2018.8516791.

Drgoňa, J., Tuor, A., Koch, J., Shapiro, M., Vrabie, D., 2023. Neuro-MANCER: Neural Modules with Adaptive Nonlinear Constraints and Efficient Regularizations URL: <https://github.com/pnnl/neuromancer>.

Kaiser, E., Kutz, J.N., Brunton, S.L., 2021. Data-driven discovery of koopman eigenfunctions for control. Machine Learning: Science and Technology 2, 035023.

Koopman, B.O., 1931. Hamiltonian systems and transformation in hilbert space. Proceedings of the National Academy of Sciences 17, 315–318.

Table A.7: CSTR series operational constraints and initial conditions.

Parameter	Description	Value/Range
$F \in$	Fresh feed flow rate	$[0.0, 2.0 \times 10^{-4}] \text{ m}^3/\text{s}$
$L \in$	Recycle flow rate	$[0.0, 2.0 \times 10^{-4}] \text{ m}^3/\text{s}$
$T_{c,1}, T_{c,2} \in$	Coolant temperatures	$[280.0, 330.0] \text{ K}$
x_0	Initial state (steady state)	Computed for $u_0 = [1.5 \times 10^{-4}, 7.5 \times 10^{-5}, 290, 290]^\top$
T_s	Sampling time	1.0 s

- Korda, M., Mezić, I., 2018. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica* 93, 149–160.
- Li, H., Swartz, C.L.E., 2019. Dynamic real-time optimization of distributed MPC systems using rigorous closed-loop prediction. *Computers & Chemical Engineering* 122, 356–371. doi:10.1016/j.compchemeng.2018.08.028.
- Liu, J., Muñoz de la Peña, D., Christofides, P.D., 2009. Distributed model predictive control of nonlinear process systems. *AIChE Journal* 55, 1171–1184.
- Lusch, B., Kutz, J.N., Brunton, S.L., 2018. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature communications* 9, 4950.
- Mezić, I., 2005. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics* 41, 309–325.
- Pannocchia, G., Rawlings, J.B., 2003. Disturbance models for offset-free model-predictive control. *AIChE journal* 49, 426–437.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. Pytorch: An imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems* 32, pp. 8024–8035.

- Rowley, C.W., Mezić, I., Bagheri, S., Schlatter, P., Henningson, D.S., 2009. Spectral analysis of nonlinear flows. *Journal of fluid mechanics* 641, 115–127.
- Van Overschee, P., De Moor, B., 2012. Subspace identification for linear systems: Theory—Implementation—Applications. Springer Science & Business Media.
- Williams, M.O., Kevrekidis, I.G., Rowley, C.W., 2015. A data-driven approximation of the koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science* 25, 1307–1346.